

SupportIQ — Multi-Agent Implementation Architecture

How AI Agents Built This Platform End-to-End

The Core Idea

SupportIQ was not written manually line by line. It was built using a **multi-agent AI development system** where specialist AI agents collaborated — each agent completing one job, passing its output to the next agent as structured input.

This is the same principle as a software team: an architect designs, developers implement in parallel, a tester writes tests, a reviewer approves. Except every role is an AI agent, the handoffs are automated, and the entire cycle runs in minutes.

Two Layers of Multi-Agent Usage

LAYER 1 – SDLC (How SupportIQ was BUILT)

Multi-agent workflows orchestrated the entire development pipeline: research → design → parallel implementation → tests → review → commit

LAYER 2 – RUNTIME (What SupportIQ DOES at production time)

AI support intelligence: classify, score, triage, draft
5 AI-powered endpoints backed by EPAM DIAL

This document covers **Layer 1** — the multi-agent system that built the platform.

Specialist Agents — Each With a Defined Role

Six dedicated AI agents were configured, each with a custom system prompt tuned to a specific domain:

Agent	Domain	What It Does
taskflow-architect	System design	Reads requirements, analyses current codebase state, produces a precise implementation plan per file
taskflow-reviewer	Code quality	Reviews changed files against 20+ Spring Boot conventions, returns structured findings with severity ratings
taskflow-tester	Test engineering	Writes JUnit 5 + MockMvc tests, knows the exact @MockBean / @WebMvcTest patterns required
support-analyst-agent	AI/LLM design	Designed the system prompts, JSON schema contracts, and sentiment/escalation scoring logic for SupportIQ
support-triage-agent	Queue management	Analysed the triage strategy, SLA implications, and urgency ranking logic for the bulk triage endpoint
demo-guide-agent	Presentation	Produced demo scripts, management talking points, and ROI framing from real API response data

Workflow 1 — Feature Implementation

How a GitHub Issue Became Production Code

Every SupportIQ feature was implemented through this 6-phase automated pipeline:

GitHub Issue #N



PHASE 1 – RESEARCH (2 agents in PARALLEL)

Agent A: mcp-read-issue

- └ Reads GitHub issue via MCP server
- └ Extracts: title, requirements, acceptance criteria as structured JSON

Agent B: read-source-files

- └ Reads all 6 current Java source files
- └ Returns current fields, methods, imports

Both agents run simultaneously →

Output: issue requirements + codebase state
(Agent A and Agent B outputs MERGED)



PHASE 2 – PLAN (1 agent, sequential)

Agent: taskflow-architect

Input: issue requirements + current codebase state

- └ Decides exactly what changes in each file

Output: structured plan per layer

```
{ entityChanges, requestDtoChanges,  
  responseDtoChanges, mapperChanges,  
  repositoryChanges, testChanges }
```

Output: implementation plan
(This plan is passed to ALL 3 parallel implementers below)



PHASE 3 – IMPLEMENT (3 agents in PARALLEL)

Agent A: impl-entity

Input: plan.entityChanges

- └ Edits Task.java – adds field, annotation, import

Agent B: impl-request-dto

Input: plan.requestDtoChanges

- └ Edits TaskRequestDTO.java – adds Jakarta validation annotation, exact message

Agent C: impl-response-dto

Input: plan.responseDtoChanges

- └ Edits TaskResponseDTO.java – adds response field

All 3 agents work simultaneously on different files

| All 3 implementations complete



PHASE 4 – INTEGRATE (2 agents in PARALLEL)
(Runs AFTER Phase 3 – depends on updated DTOs)

Agent A: impl-mapper

Input: plan.mapperChanges

└ Edits TaskMapper.java – maps new field in
toResponseDTO, toEntity, updateEntityFromDTO

Agent B: impl-repository

Input: plan.repositoryChanges

└ Edits TaskRepository.java – adds query methods

| Integration complete



PHASE 5 – TEST (2 agents in PARALLEL)

Agent A: taskflow-tester (service tests)

Input: plan.testChanges + issue.acceptanceCriteria

└ Updates TaskServiceTest.java
└ Adds new field to ALL builder calls

Agent B: taskflow-tester (controller tests)

Input: issue.acceptanceCriteria (exact assertions)

└ Updates TaskControllerValidationTest.java
└ Adds @WebMvcTest validation test per criterion

| Tests written



PHASE 6 – REVIEW (1 agent, sequential)

Agent: taskflow-reviewer

Input: issue requirements + acceptance criteria

└ Reads ALL changed files
└ Verifies every requirement is implemented
└ Verifies every criterion has a test
└ Checks conventions (@MockBean, no @Autowired)

Output: { readyToCommit: true/false,
 conventionViolations: [...] }



git commit "feat: <title> closes #N"

Agent communication: Each phase passes its output as direct input to the next. The architect's plan flows into 5 parallel implementers. The issue's acceptance criteria flow directly into the test agent's assertions. The final reviewer receives both the original requirements AND reads the actual code — so it can verify whether they match.

Workflow 2 — Code Review

4 Specialist Agents Reviewing Simultaneously

Before any code was committed to the SupportIQ feature branch, a parallel code review workflow ran:



Workflow 3 — Support AI Insights

4 Parallel Analysts + Executive Briefing

This workflow runs against the **live SupportIQ queue** and produces a management-level intelligence briefing:



Returns: structured queue summary

Queue data

(Same data passed to ALL 4 analysts below)



PHASE 2 – ANALYZE (4 agents in PARALLEL)

Each agent receives the SAME queue data
but analyses a different dimension:

Agent 1: Queue Health & Operations

└ Open/resolved ratio, escalation rate,
IN_PROGRESS activity

└ Score: 0-100

Agent 2: Escalation & Churn Risk

└ Highest churn risk customers, legal threats,
systemic patterns, financial impact

└ Score: 0-100

Agent 3: Customer Sentiment & Experience

└ Emotional state of customer base, angry vs
satisfied distribution, NPS prediction

└ Score: 0-100

Agent 4: Issue Patterns & Root Cause

└ Dominant categories, product/process signals,
self-service opportunities

└ Score: 0-100

4 scores + 4 finding sets + urgent actions

(All 4 outputs MERGED before next phase)



PHASE 3 – SYNTHESIZE (1 agent, sequential)

Input: 4 dimension scores + all findings merged

└ Writes an executive briefing a VP reads
in 2 minutes

Output: overallHealthScore, businessRisks,
wins, topPriorityActions

How Agents "Talk" to Each Other

The agents do not communicate directly in real time. Instead they communicate through **structured data handoffs** — the output of one agent is the input of the next.

Agent A produces:

```
{
  "requirements": ["Add dueDate field", "Validate: past dates rejected"],
  "acceptanceCriteria": ["POST with past date returns 400", "Message: Due date must be..."]
}
```

|
| This exact JSON is passed into Agent B's prompt



Agent B (architect) reads the requirements and produces:

```
{
  "entityChanges": ["Add LocalDate dueDate field with @Column(nullable=true)"],
  "requestDtoChanges": ["Add @FutureOrPresent with message 'Due date must be today...'",],
  "testChanges": ["Add pastDate rejection test in controller test"]
}
```

|
| This exact plan is passed into 3 parallel implementers
▼

Agent C, D, E each implement their assigned file simultaneously

This is structured agent communication — each agent's response is a typed contract that the orchestrator passes forward. No agent needs to "know" about the others. The workflow is the coordinator.

Results

Metric	Value
Total specialist agents	6
Workflows orchestrated	5
Parallel agent phases across all workflows	8
Test suite size	103+ tests
Lines of production Java	~2,000+
Time to build SupportIQ (all 5 endpoints + UI + agents + workflows)	Day 8 of the series

Key Takeaway for Architects

The multi-agent pattern used here is a **production-grade AI SDLC**, not a prototype. Every agent has:

- A defined role and system prompt
- A typed input schema (what it receives)
- A typed output schema (what it returns)
- A single responsibility — no agent does more than one job

The orchestration (what runs in parallel, what waits, what feeds forward) is deterministic JavaScript — not model-driven. The models handle reasoning within their scope; the workflow handles coordination across scopes.

This is the **AI-native engineering pattern**: use AI not just in the product, but in the process of building the product.

Multi-agent SDLC implemented using Claude Code workflows — Day 8, AI-Native Engineering Series